# Laboratorio S1 · 135 minutos

Se trabaja dentro de `04-Proyecto-Base/` con el entorno `env-ia-mlops` (terminal: **Anaconda Prompt** o la terminal de VS Code con el entorno activado). Los comandos valen en CMD y PowerShell. `make` no existe en Windows: se usan las líneas equivalentes.

| Min | Bloque |
|---|---|
| 0–10 | 0. Preparar |
| 10–40 | 1. Cuadernos 00 y 01: operacional, RAW → MASTER |
| 40–65 | 2. Cuaderno 02: MASTER → FS |
| 65–75 | **Pausa** |
| 75–100 | 3. Cuaderno 03: FS → ML |
| 100–125 | 4. Cuaderno 04: entrenar y MLflow |
| 125–135 | 5. Traducir a `.py` y probar |

Los números que verás con la **muestra** (23 070 filas) son distintos de los del dato completo de las diapositivas. Es esperado.

## 0. Preparar (10 min)

```
conda activate env-ia-mlops
cd 04-Proyecto-Base
python -m pytest -q -m "not lento"
```

Esperado: `127 passed` (las 4 pruebas restantes ejecutan los cuadernos y son lentas). En VS Code abre cada `.ipynb` y elige el kernel **env-ia-mlops**.

## 1. Cuadernos 00 y 01 · operacional, RAW → MASTER (30 min)

Abre `notebooks/00_pipeline_operacional.ipynb`: su introducción explica todo el recorrido. Ejecútalo (parámetro `FUENTE = 'muestra'`). **Mira:** el diagrama E-R de `retail.db`, la comprobación «un cliente y un país por factura» y que se reconstruyen las 23 070 líneas.

Después abre `notebooks/01_pipeline_raw_a_master.ipynb` y ejecuta celda a celda. **Mira:**

- La ingesta a RAW: cuatro tablas `raw_*` copiadas del operacional y `raw_carga` con filas y huella de cada una (23 070 líneas).
- `master_descartes`: cuántas filas y con qué motivo.
- La conciliación: `raw = master_lineas + descartes`.
- `master_calidad`: 13 reglas, ninguna en `ERROR`; con la muestra, R06 (facturas con líneas en fechas distintas) da `ALERTA` con valor 1.

**Preguntas:** ¿cuántas líneas no tienen cliente? ¿Por qué no se descartan? ¿Por qué R06 es una alerta y no un error? **Comprobación:** `SELECT COUNT(*) FROM raw_lineas_factura;` da 23 070 en el visor SQLite.

## 2. Cuaderno 02 · MASTER → FS (25 min)

Ejecuta `notebooks/02_pipeline_master_a_fs.ipynb`. **Mira:** `fs_catalogo` (13 variables), una fila de `fs_cliente_corte` y el recálculo manual con pandas del cliente de ejemplo. **Haz:** cambia el corte de ejemplo a otra fecha y comprueba que la recencia cambia como esperas. **Prueba definitiva** (terminal):

```
python -m pytest tests -k futuro -q
```

Esperado: `2 passed` (todas las pruebas cuyo nombre contiene «futuro»). Esta prueba borra todo lo posterior al corte y comprueba que las variables no cambian.

## Pausa (10 min)

## 3. Cuaderno 03 · FS → ML (25 min)

Ejecuta `notebooks/03_pipeline_fs_a_ml.ipynb`. **Mira:** `ml_particiones` (ventanas que no se solapan), el número de filas y la tasa de recompra por partición (con la muestra: train 626 filas · 50,8 %, valid 114 · 45,6 %, test 250 · 40,4 %). **Pregunta:** ¿por qué la tasa baja de train a test? ¿Qué implicaría para un modelo en producción?

## 4. Cuaderno 04 · Entrenar y MLflow (25 min)

Ejecuta `notebooks/04_entrenamiento_y_seguimiento.ipynb`. **Mira:** la referencia (AUC 0,5), regresión logística y gradient boosting (con la muestra: regresión logística valid 0,698 · test 0,747; gradient boosting 0,663 · 0,735). Después, desde una terminal en `04-Proyecto-Base`:

```
python -m ia_mlops.mlflow_ui
```

Antes de abrir nada dice qué registro lee y cuántas ejecuciones y versiones tiene. Abre http://127.0.0.1:5000 y:

1. Arriba a la izquierda, elige **Model training** (no *GenAI*).
2. Entra al experimento `retencion-90d`, selecciona **dos ejecuciones** y pulsa *Compare*. **Anota:** la huella del dato (`huella_datos`) de ambas. ¿Coincide? Solo entonces la comparación es válida.
3. En el menú izquierdo, **Model registry** → `retencion-90d`: abre la última versión y mira de qué ejecución salió (*Source Run*) y sus etiquetas.

**Por qué no `mlflow ui --backend-store-uri sqlite:///models/mlflow.db` a secas:** la ruta es relativa. Desde otra carpeta (la terminal de VS Code se abre en la raíz del repositorio), MLflow crea ahí una base **vacía** sin avisar.

## 5. Traducir a `.py` y probar (10 min)

```
python -m ia_mlops.pipeline                # los cuatro pipelines .py: ingesta_raw,
ingesta_master, ml, train
python -m ia_mlops.pipeline                # otra vez: los cuatro dicen «sin cambios»
(idempotencia)
python -m ia_mlops.pipeline_train --forzar  # uno solo, forzado
jupytext --to py:percent notebooks/01_pipeline_raw_a_master.ipynb -o
scripts_desde_notebooks/01_pipeline_raw_a_master.py
python -m pytest tests/test_notebooks.py -q
```

(`jupytext` convierte un cuaderno a la vez; repite con los otros si hay tiempo. La prueba de equivalencia puede tardar unos minutos.) **Preguntas:** ¿qué demuestra la prueba de equivalencia? ¿Por qué la segunda ejecución del pipeline no vuelve a entrenar ni añade ejecuciones a MLflow?

## Si algo falla

| Síntoma | Qué hacer |
|---|---|
| `ModuleNotFoundError` en un cuaderno | Kernel equivocado: selecciona `env-ia-mlops`. |
| El 02 falla al leer tablas | Ejecuta primero el 01 completo. |
| `No module named ia_mlops` al abrir MLflow | No estás en `04-Proyecto-Base`: `cd 04-Proyecto-Base` y repite. |
| MLflow abre pero no hay ejecuciones ni modelos | `python -m ia_mlops.mlflow_ui --comprobar`: si da 0, ejecuta el cuaderno 04 con `USAR_MLFLOW = True`. Si ves *Traces* o *Prompts*, cambia a **Model training**. |
| El puerto 5000 está ocupado | `python -m ia_mlops.mlflow_ui --port 5001` y abre http://127.0.0.1:5001. |
| La prueba de equivalencia falla | Anota el mensaje en las evidencias; no lo ocultes. |

## Para quien termina antes

Empieza la tarea: `../tarea/README.md`.